

Wisdom & Chance TCG — Mobile App Server Sync Instructions

Connecting the Mobile App to the New Server & Database

****Prepared: March 17, 2026****

OVERVIEW

The Wisdom & Chance web server has been migrated to a new Replit personal account. The mobile app needs to be updated to connect to this new server. The server provides:

- REST API for all game data (cards, decks, users, achievements, leaderboard, etc.)
 - WebSocket server for real-time multiplayer, chat, and presence
 - JWT authentication for mobile clients
 - PostgreSQL database with all existing user data, cards, and artwork
-

STEP 1: UPDATE THE SERVER URL

Old URL (no longer active):

```
https://wisdom-and-chance.replit.app
```

New URL (use this everywhere):

****Check your Replit deployment for the exact URL**** — it will be something like:

```
https://YOUR-NEW-APP-NAME.replit.app
```

Files to Update

Search the entire mobile app codebase for any reference to the old URL and replace it with the new one. Common locations:

1. ****API base URL / HTTP client configuration**** — wherever the base URL for REST API calls is defined
2. ****WebSocket connection URL**** — change from `wss://OLD-URL/ws` to `wss://NEW-URL/ws`
3. ****Image URLs**** — card artwork URLs that reference the old server domain
4. ****Environment/config files**** — `.env`, config constants, or build configuration

Search command:

```
grep -r "wisdom-and-chance" --include="*.ts" --include="*.tsx" --include="*.js" --include="*.json" --include="*.env" .
```

Replace ALL occurrences with the new deployed URL.

STEP 2: VERIFY AUTHENTICATION (JWT)

The mobile app uses JWT (JSON Web Token) authentication. The flow has NOT changed:

Login Flow:

```
POST https://NEW-URL/api/mobile/auth/login
Content-Type: application/json
```

```
{
  "email": "user@example.com",
  "firstName": "John",           // optional
  "lastName": "Doe",            // optional
  "profileImageUrl": "https://...", // optional
  "provider": "google",          // optional: "google" or "apple"
  "providerToken": "..."       // optional: for server-side verification
}
```

Response:

```
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": "uuid-string",
    "email": "user@example.com",
    "firstName": "John",
    "lastName": "Doe",
    "profileImageUrl": "https://..."
  }
}
```

Using the Token:

Include the JWT in ALL subsequent API requests:

```
Authorization: Bearer eyJhbGciOi...
```

Token Refresh (before expiry — tokens last 7 days):

```
POST https://NEW-URL/api/mobile/auth/refresh
Authorization: Bearer CURRENT_TOKEN
```

Returns: { "token": "new-jwt-token" }

Get Current User:

```
GET https://NEW-URL/api/mobile/auth/me
Authorization: Bearer TOKEN
```

STEP 3: VERIFY WEBSOCKET CONNECTION

The WebSocket server is at path /ws on the same domain.

Connection URL for Mobile (with JWT):

```
wss://NEW-URL/ws?token=YOUR_JWT_TOKEN
```

Connection Flow:

1. Connect to the WebSocket URL with token query parameter
2. Receive auth_success message: { type: "auth_success", payload: { userId, displayName } }

3. If auth fails, receive `auth_error: { type: "auth_error", payload: { message } }`
4. Once authenticated, send/receive game events

Message Format (all messages):

```
{ "type": "event_name", "payload": { ... } }
```

Client-to-Server Events:

Event	Payload	Description
join_room	{ roomId }	Subscribe to room updates
leave_room	{ roomId }	Unsubscribe from room
join_game	{ gameId }	Subscribe to game state
leave_game	{ gameId }	Unsubscribe from game
room_message	{ roomId, message }	Send chat in room
game_message	{ gameId, message }	Send chat in game
game_action	{ gameId, action }	Send game action (card play, attack, etc.)

Server-to-Client Events:

Event	Payload	Description
auth_success	{ userId, displayName }	Connection authenticated
auth_error	{ message }	Auth failed
room_update	Room state object	Room state changed
player_joined	{ roomId, userId }	Player joined room
player_left	{ roomId, userId }	Player left room
player_ready_update	{ roomId, userId, ready, deckId }	Ready status changed
game_start	{ roomId, gameId }	Game started
game_action	Game action object	Game state update
chat_message	{ roomId, senderId, message, createdAt }	Room chat
game_message	{ userId, displayName, message, timestamp }	In-game chat
friend_message	{ id, senderId, receiverId, message, createdAt }	Direct message
friend_request	{ requestId, senderId }	Incoming friend request
friend_request_accepted	{ requestId, friendId }	Request accepted
presence_update	{ userId, status: "online"/"offline" }	Friend online status
spectator_joined	{ roomId, userId }	Spectator joined
spectator_left	{ roomId, userId }	Spectator left

STEP 4: COMPLETE API REFERENCE

All endpoints use the base URL: `https://NEW-URL`
All protected endpoints require: Authorization: Bearer TOKEN

Health Check (No Auth Required)

```
GET /api/health
```

Response: { status: "ok", timestamp, version, database: "connected", services: { auth, websocket, multiplayer } }

****Use this endpoint to verify connectivity before anything else.****

PUBLIC ENDPOINTS (No Auth Required)

Cards

Method	Path	Description
GET	/api/cards	List all cards
GET	/api/cards/:id	Get single card by ID
GET	/api/cards/element/:element	Get cards by element (fire/water/earth/air/nature)

Commanders

Method	Path	Description
GET	/api/commanders	List all commanders
GET	/api/commanders/:id	Get single commander by ID

Achievements

Method	Path	Description
GET	/api/achievements	List all achievements

Leaderboard

Method	Path	Description
GET	/api/leaderboard	Top 100 players by ELO rating
		Tier values: Bronze (<800), Silver (<1000), Gold (1600+)

Daily Challenges

Method	Path	Description
GET	/api/daily-challenges	Get today's challenges

Rooms (Public Listing)

Method	Path	Description
GET	/api/rooms	List public rooms in "waiting" status
GET	/api/rooms/:id	Get room details with host/guest/spectators
GET	/api/rooms/:id/messages	Get room chat (last 100 messages)

**PROTECTED
ENDPOINTS (Auth
Required)**

User Profile

Method	Path	Description
PATCH	/api/user/profile	Update profile (firstName, lastName, profileImageUrl)

Decks

Method	Path	Description
GET	/api/user-decks	List user's saved decks
GET	/api/user-decks/:id	Get specific deck
POST	/api/user-decks	Save new deck (name, commanderId, cardIds[40])
PATCH	/api/user-decks/:id	Update deck
DELETE	/api/user-decks/:id	Delete deck

****Deck Validation Rules:****

- Exactly 40 cards
- 4 cards per power rank (1-10)
- Max 3 copies of any single card
- Must have a valid commander ID

AI Deck Suggestions

Method	Path	Description
POST	/api/deck-suggestions	AI-generated deck (body: { commanderId, playstyle: "aggressive"/"defensive"/"balanced" })

Friends

Method	Path	Description
GET	/api/friends	List friends with online status
GET	/api/friend-requests	List pending requests (incoming + outgoing)
POST	/api/friend-requests	Send request (body: { email })
POST	/api/friend-requests/:id/accept	Accept request
POST	/api/friend-requests/:id/decline	Decline request
DELETE	/api/friends/:friendId	Remove friend

**Friend Messages (24-hour
expiry)**

Method	Path	Description
GET	/api/friend-messages/:friendId	Get messages (last 24h, max 100)
POST	/api/friend-messages/:friendId	Send message (body: { message })

Rooms (Actions)

Method	Path	Description
POST	/api/rooms	Create room (body: { name, isPrivate?, password?, gameMode? })
POST	/api/rooms/:id/join	Join as player (body: { password? })
POST	/api/rooms/:id/leave	Leave room
POST	/api/rooms/:id/ready	Set ready (body: { ready?, deckId })
POST	/api/rooms/:id/start	Start game (host only, both must be ready)
POST	/api/rooms/:id/spectate	Join as spectator
DELETE	/api/rooms/:id/spectate	Leave spectator mode
POST	/api/rooms/:id/messages	Send chat message (body: { message })

Player Stats & Ratings

Method	Path	Description
GET	/api/player-stats	Get user's game stats (auto-creates if new)
GET	/api/player-rating	Get user's ELO rating (auto-creates at 1000)
GET	/api/player-achievements	Get user's unlocked achievements
GET	/api/player-challenges	Get user's challenge progress
POST	/api/player-challenges/:id/claim	Claim completed challenge reward

User Search

Method	Path	Description
GET	/api/users/search?q=QUERY	Search users by email/name (max 10 results)

STEP 5: DATA MODELS

Card

```
{
  "id": "uuid",
  "name": "string",
  "element": "fire | water | earth | air | nature",
  "power": 1-10,
  "trait": "string | null",
  "traitValue": "number | null",
  "buffModifier": 0,
  "buffColor": "string | null",
  "debuffModifier": 0,
  "debuffColor": "string | null",
  "description": "string | null",
  "imageUrl": "string | null",
  "isCommander": false
}
```

Commander

```
{
  "id": "uuid",
  "name": "string",
  "element": "string",
  "ability": "string",
  "abilityDescription": "string",
  "imageUrl": "string | null"
}
```

User

```
{
  "id": "uuid",
  "email": "string",
  "firstName": "string | null",
  "lastName": "string | null",
  "profileImageUrl": "string | null",
  "createdAt": "ISO 8601",
  "updatedAt": "ISO 8601"
}
```

UserDeck

```
{
  "id": "uuid",
  "userId": "uuid",
  "name": "string",
  "commanderId": "uuid",
  "cardIds": ["uuid", "uuid", "...40 total"],
  "createdAt": "ISO 8601",
  "updatedAt": "ISO 8601"
}
```

PlayerStats

```
{
  "id": "uuid",
  "userId": "uuid",
  "totalXp": 0,
  "level": 1,
  "gamesPlayed": 0,
  "gamesWon": 0,
  "gamesLost": 0,
  "totalDamageDealt": 0,
  "totalCardsPlayed": 0,
  "favoriteElement": "string | null",
  "favoriteCommander": "string | null",
  "longestWinStreak": 0,
  "currentWinStreak": 0,
  "updatedAt": "ISO 8601"
}
```

PlayerRating

```
{
  "id": "uuid",
  "userId": "uuid",
  "rating": 1000,
  "wins": 0,
  "losses": 0,
  "streak": 0
}
```

STEP 6: GAME RULES (for reference)

- **Deck size:** 40 cards (4 per power rank 1-10) + 1 commander
- **Starting HP:** 20
- **Starting hand:** 5 cards
- **Cards drawn per turn:** 2
- **Cards deployed per turn:** 2
- **Elements:** Fire, Water, Earth, Air, Nature
- **Turn phases:** Draw, Deployment, Combat, Calculation, End
- **Victory:** Reduce opponent HP to 0, or opponent has no cards in deck and hand
- **ELO tiers:** Bronze (<800), Silver (<1000), Gold (<1200), Platinum (<1400), Diamond (<1600), Master (1600+)

STEP 7: QUICK CONNECTIVITY TEST

Tell the mobile app AI to run these checks in order:

1. **Health check:** GET https://NEW-URL/api/health — should return { status: "ok" }
 2. **Login:** POST https://NEW-URL/api/mobile/auth/login with { "email": "test@example.com" } — should return a token
 3. **Cards:** GET https://NEW-URL/api/cards — should return the full card list
 4. **WebSocket:** Connect to wss://NEW-URL/ws?token=TOKEN — should receive auth_success
- If all four pass, the mobile app is fully synced with the server.

STEP 8: TELL THE MOBILE APP AI

Copy and paste this to the mobile app AI agent:

"Update the server URL from the old domain to NEW-URL.replit.app everywhere in the codebase. This includes: 1. API base URL for all HTTP requests 2. WebSocket connection URL (wss://) 3. Any image URLs that reference the server 4. Environment/config files The API endpoints, authentication flow (JWT), and WebSocket protocol have NOT changed. Only the domain has changed. Run the health check at /api/health to verify connectivity after updating."

LIVE API DOCUMENTATION

The server provides a machine-readable API reference at:

```
GET https://NEW-URL/api/docs
```

This returns the complete API documentation as JSON, which the mobile app can use for auto-discovery.